

Filière : SDBDIA 2ème année

Fine-tuning d'un Small Language Model pour la Génération de Résumés à partir de Données Tabulaires/textuel

Rapport d'avancement de stage 2

Réalisé par:

Youssef Aitbouddroub

1. Contexte et objectif de cette étape

Suite au fine-tuning réussi de FLAN-T5 Base avec LoRA (rapport précédent), l'étape suivante du stage consistait à expérimenter QLoRA (Quantized LoRA), une variante qui combine LoRA avec une quantification 4 bits du modèle de base. L'objectif est de réduire la consommation de mémoire GPU, ce qui permettrait théoriquement d'entraîner plus vite (par exemple avec une taille de lot plus grande) ou d'entraîner des modèles plus gros sur le même matériel.

Cette étape visait à vérifier si cet avantage se confirme concrètement sur notre cas d'usage : un modèle de taille modérée (FLAN-T5 Base, 250 millions de paramètres) entraîné sur un GPU grand public (RTX 3060, 12 Go de VRAM).

2. Configuration QLoRA

La configuration LoRA (rang, modules ciblés, taux d'apprentissage) a été conservée à l'identique par rapport à l'étape précédente, seule la représentation numérique du modèle de base change :

- Quantification : 4 bits, type NF4 (Normal Float 4), avec double quantification activée (quantification imbriquée pour économiser encore un peu de mémoire)
- Calculs effectués en bfloat16 (les poids quantifiés sont décompressés à la volée pour chaque calcul)
- Paramètres LoRA : rang $r = 8$, $\alpha = 16$, modules q/v ; identiques à l'étape LoRA
- Paramètres entraînaables : 884 736 sur 248 millions (0,36 %), comme précédemment
- Taux d'apprentissage : 1×10^{-4} , 3 époques, même matériel (RTX 3060, 12 Go)

Une différence de configuration est à signaler : le nombre de pas d'accumulation de gradient (`gradient_accumulation_steps`) a été changé de 2 (LoRA) à 1 (QLoRA) au cours des essais. Cette différence n'est pas liée à QLoRA lui-même et introduit un facteur de comparaison supplémentaire, discuté en section 4.

3. Résultats de l'entraînement

L'entraînement s'est déroulé sans problème numérique (aucun NaN), grâce aux corrections déjà validées lors de l'étape LoRA (précision bfloat16 cohérente, échelle LoRA modérée). Les deux premières époques ont donné les pertes suivantes :

Époque	Perte d'entraînement	Perte de validation
1	1.136	1.614
2	1.028	1.575
3	1.123	1.569

À la fin des 3 époques (22 644 pas au total, contre 11 322 pour LoRA conséquence directe du changement de `gradient_accumulation_steps`), l'entraînement complet a duré environ 3h, pour une perte d'entraînement moyenne globale de 1.82.

4. Analyse : QLoRA a-t-il réduit le temps ou les ressources nécessaires ?

C'est la question centrale de cette étape. Plusieurs expériences ont été menées pour y répondre, avec des résultats systématiquement négatifs, ce qui constitue en soi une conclusion utile et argumentée.

4.1 Vitesse d'entraînement

La quantification 4 bits n'accélère pas l'entraînement : elle exige au contraire de décompresser (« déquantifier ») chaque poids en bfloat16 à chaque passage avant et arrière, ce qui ajoute du calcul. Les cartes graphiques grand public comme la RTX 3060 ne disposent pas non plus d'accélération matérielle dédiée au calcul en 4 bits. Résultat observé : la vitesse par pas (~2,2 it/s) et la durée totale d'entraînement sont équivalentes à LoRA, sans aucun gain.

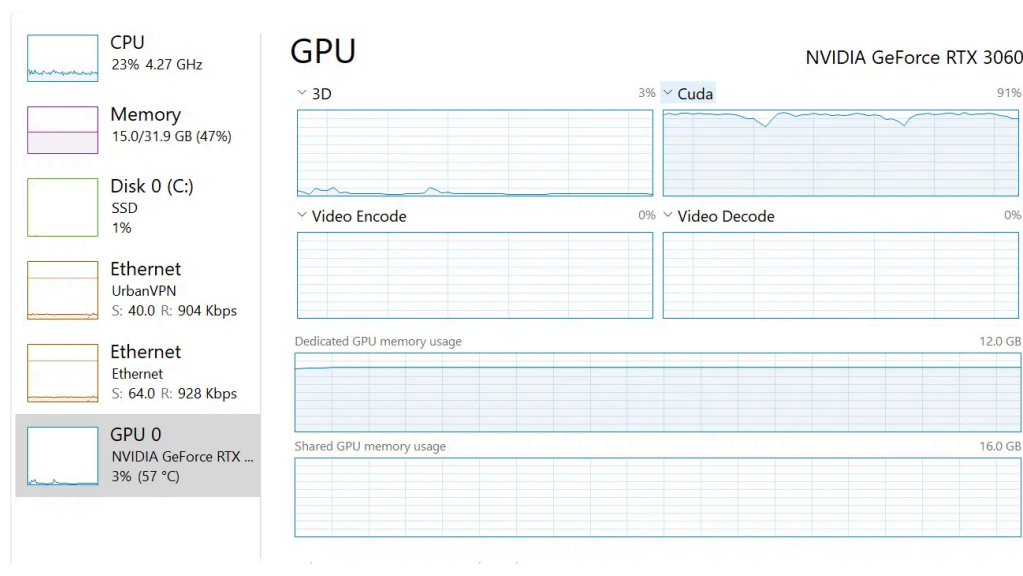
4.2 Marge de mémoire GPU disponible

L'hypothèse testée principale de QLoRA est de libérer de la mémoire GPU, permettant par exemple d'augmenter la taille de lot (batch size) ou la longueur maximale des séquences. Plusieurs essais ont été réalisés :

- Augmentation de la taille de lot de 16 à 24 ou 32 : temps estimé (ETA) explosant à environ **16 heures**
- Augmentation de la longueur maximale d'entrée de 384 à 512 tokens : même problème, ETA à **17 heures**

Le gestionnaire des tâches Windows a permis de confirmer la cause : à taille de lot 16 (configuration de base), la mémoire GPU dédiée est déjà utilisée à presque 100 % (avec utilisation CUDA à 91 %). Dès qu'on demande plus de mémoire, Windows bascule automatiquement vers la « mémoire GPU partagée », c'est-à-dire la RAM système accessible via le bus PCIe ; beaucoup plus lente que la mémoire dédiée de la carte graphique. C'est ce basculement qui provoque l'effondrement des performances.

Explication technique : la quantification 4 bits ne réduit que la taille des poids du modèle de base gelé (environ 500 Mo en bfloat16, ramenés à environ 125 Mo en 4 bits, soit ~375 Mo économisés). Or, pour ce modèle de taille modérée (250M paramètres), la mémoire est en réalité dominée par les activations intermédiaires (qui dépendent de la taille de lot et de la longueur des séquences), et non par les poids du modèle. Les ~375 Mo économisés sont donc négligeables face aux plusieurs Go nécessaires pour augmenter le lot ou la longueur des séquences.



4.3 Optimiseurs « paged » (paged_adamw_32bit / paged_adamw_8bit)

Une deuxième piste a été testée : les optimiseurs « paged », conçus pour déplacer automatiquement l'état de l'optimiseur vers la RAM système en cas de manque de mémoire GPU. Résultat : les temps estimés sont passés à 34 heures (paged_adamw_32bit) et 27 heures (paged_adamw_8bit) bien pires que la configuration de base.

Deux raisons expliquent cet échec : d'une part, avec seulement 884 736 paramètres entraînaibles (LoRA), l'état de l'optimiseur ne représente que quelques Mo il n'y avait donc aucun problème de mémoire à résoudre. D'autre part, la gestion de la mémoire unifiée CUDA est connue pour être particulièrement inefficace sous Windows (pilote WDDM), ce qui pénalise fortement ces optimiseurs même lorsqu'aucun transfert réel vers la RAM n'est nécessaire. La configuration a été ramenée à l'optimiseur standard **adamw_torch**.

5. Comparaison de la qualité : LoRA vs QLoRA

Au-delà des ressources, il restait à vérifier si la quantification 4 bits dégrade la qualité des résumés générés. Les deux modèles fine-tunés (LoRA et QLoRA) ont été évalués dans les mêmes conditions (mêmes paramètres de génération, même échantillon de validation) :

Métrique	Modèle de base	Fine-tuné LoRA	Fine-tuné QLoRA
ROUGE-L	0.191	0.377	0.382
BLEU	0.046	0.171	0.171
METEOR	0.211	0.400	0.411
BERTScore-F1	0.856	0.903	0.904

Les scores de LoRA et QLoRA sont quasiment identiques (écarts de 0,05 % à 1,1 % selon la métrique), largement dans la marge de variation normale d'une exécution à l'autre (aléas du décodage par recherche en faisceau, entre autres). Ce résultat est en réalité conforme à la théorie : la quantification est une opération qui ne peut que faire perdre de l'information sur les poids, jamais en ajouter elle ne peut donc pas améliorer la qualité du modèle. Le fait d'observer une égalité quasi parfaite indique que la perte d'information causée par la quantification 4 bits est négligeable à cette échelle de modèle, plutôt qu'un avantage réel de QLoRA.

Le changement de gradient_accumulation_steps (2 pour LoRA, 1 pour QLoRA) n'a pas été neutralisé côté entraînement ; il reste une variable non contrôlée entre les deux expériences. Les écarts de résultats observés restent toutefois trop faibles pour être attribués à cette différence plutôt qu'à la variance normale d'exécution.

6. Conclusion

Sur ce cas d'usage précis un modèle de taille modérée (250M de paramètres) entraîné sur un GPU grand public déjà saturé en mémoire dès la configuration de base QLoRA n'apporte aucun bénéfice mesurable :

- Aucun gain de vitesse d'entraînement (durée équivalente à LoRA)
- Aucune marge mémoire réellement exploitable (les économies portent sur les poids gelés, marginales face aux activations qui dominent la consommation mémoire ici)
- Une qualité de sortie équivalente à LoRA, sans dégradation mesurable mais sans amélioration non plus
- Les optimiseurs « paged », censés exploiter ce type d'économie mémoire, se sont révélés activement contre-productifs sous Windows

Ce résultat s'explique : les gains de QLoRA sont proportionnels à la taille du modèle de base, et ne deviennent intéressants que pour des modèles nettement plus gros (plusieurs milliards de paramètres) où les poids gelés représentent une part dominante de la mémoire utilisée. Pour FLAN-T5 Base, LoRA en bfloat16 reste le choix le plus adapté : même temps d'entraînement, même qualité, sans la complexité supplémentaire ni le risque de perte de précision liés à la quantification.